Scaling Draft Length in Speculative Decoding: Acceptance-Latency Trade-offs Across EAGLE-3, DFlash, and DSpark

Artem Shamro
Faculty of Computer Science, HSE, Moscow
adshamro@edu.hse.ru

Last Edit Date: October 27, 2026

***Abstract*—We compare three recent speculative decoding methods: EAGLE-3, DFlash, and DSpark, using the common infrastructure and released checkpoints provided by the DeepSpec repository. We partially reproduce previously reported results and study the trade-off between accepted length and decoding latency as the draft sequence length varies. At the draft length used during training, our measurements closely reproduce the results reported for DSpark, including its advantage over both DFlash and EAGLE-3. However, when evaluating beyond the training draft length, we observe substantially stronger degradation in accepted length than reported in the original DSpark study. We also find that, at batch size 1, the latency overhead of DSpark relative to DFlash is considerably larger than previously reported at large batch sizes. Despite this overhead, DSpark achieves the best acceptance–latency trade-off among the evaluated methods and outperforms the alternatives across all tested draft lengths. Finally, our results suggest that the optimal draft length is influenced by both the drafter's training horizon and the evaluation data distribution. The latency-optimal draft length varies across datasets even within the math domain, indicating that no single fixed draft length is optimal across workloads.**

The DeepSpec fork used in this work, together with the code required to reproduce our experiments and the collected experimental data, is publicly available on GitHub.[1]

Introduction

Modern LLM training increasingly relies on reinforcement learning, where a substantial fraction of computational cost is spent on inference during rollout generation. Accelerating inference is therefore an important practical objective. Speculative decoding is a rapidly developing direction that addresses this problem.

In general, the main idea is to split generation between two models of different sizes: a smaller draft model proposes candidate tokens, while the larger target model verifies them [1], [2]. In recent years, a variety of modern speculative decoding approaches have been proposed, ranging from autoregressive draft models to fully parallel diffusion-based drafters and hybrid semi-autoregressive architectures.

In this work, we partially reproduce results reported for EAGLE-3, DFlash, and DSpark [3], [4], [5] and evaluate all three methods under a common experimental setup. We focus on two characteristics of speculative decoding: the dependency of accepted length on draft length and the dependency of decoding latency on draft length. Our central research question is therefore:

Which speculative decoding approach provides the best trade-off between accepted length and latency as the draft length increases?

The experiments are based on DeepSpec [6], a full-stack framework that provides training and evaluation implementations for EAGLE-3, DFlash, and DSpark under a common codebase.

Related Work

Speculative decoding has been actively developed since the introduction of *Fast Inference from Transformers via Speculative Decoding*, which formalized lossless acceleration using a lightweight draft model followed by parallel verification by the target model [1]. Early variants explored different ways of distributing generation between models: *Big Little Decoder* used the draft model's confidence to decide when to stop small-model generation and defer to the large model, while *Medusa* replaced a separate drafter with multiple lightweight prediction heads attached to the target model [7], [8]. These approaches established the main design space of speculative decoding: improving draft quality while keeping the cost of proposing multiple future tokens substantially lower than target-model decoding.

EAGLE-3. Building on the idea of lightweight target-conditioned drafting, EAGLE introduced feature-level autoregression, where a small draft model predicts future target-model features that are subsequently mapped to token predictions [9]. EAGLE-2 further improved this framework by replacing static draft sequences with context-aware dynamically constructed draft trees [10]. EAGLE-3 shifted from feature prediction to direct token prediction [3]. In addition, EAGLE-3 fuses hidden states from multiple target-model layers and employs a training-time test procedure that better matches the multi-step drafting behavior encountered during inference. Despite its high acceptance rate, drafting

[1] https://github.com/ArtemShamro/DeepSpec

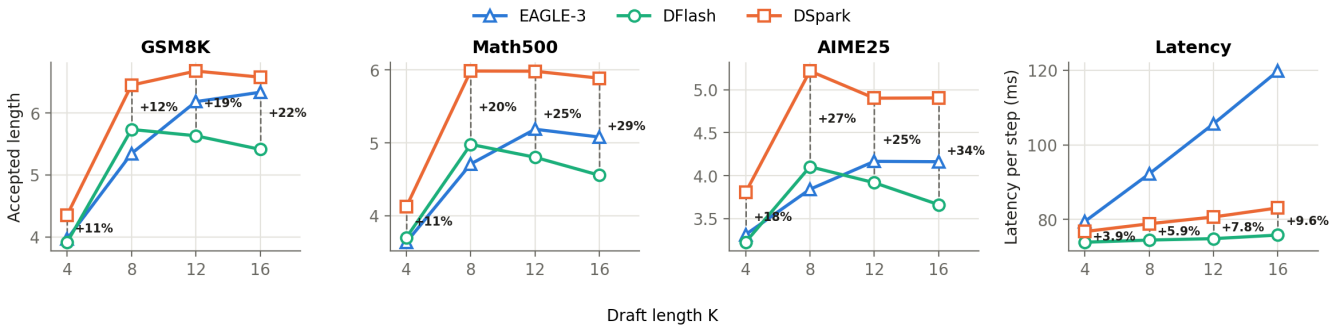


Figure 1. Effect of draft length K. Left three panels: accepted length (including the bonus token) on GSM8K, Math500 and AIME25. Right panel: latency per step, i.e. total generation time (excluding target model prefill) divided by the number of decoding steps, averaged over the three datasets. Annotations give the relative difference between DSpark and DFlash at the same K. All drafters were trained with 7 draft tokens.

remains autoregressive and therefore requires several sequential draft-model steps.

DFlash. DFlash addresses this sequential drafting bottleneck by replacing the autoregressive drafter with a lightweight block-diffusion model that predicts an entire block of candidate tokens in parallel [4]. To preserve strong conditioning on the target model, hidden representations extracted from several target layers are fused into compact context features. Instead of providing these features only at the input, DFlash injects them into the key and value projections of every draft-model layer, allowing each layer to directly attend to target-model context. This combination of one pass parallel block generation and KV injection substantially reduces drafting latency while maintaining competitive acceptance lengths.

Further analysis of DFlash revealed an important failure mode: when a draft token is rejected, the correct next token may already be present elsewhere in the proposed block, but at a different position. This behavior stems from the parallel and independent prediction of draft tokens, which lacks causal conditioning between positions [5].

DSpark. DSpark addresses this limitation by extending the DFlash-style backbone with a lightweight autoregressive component that restores inter-token dependencies [5]. Specifically, a low-rank Markov head conditions each draft position on the previously generated token, mitigating the degradation in acceptance for later positions. Another improvement introduced in DSpark is a separate confidence head that estimates token acceptance probabilities and dynamically determines how much of the proposed draft block should be verified by the target model. This reduces unnecessary target-model computation and, consequently, the overall generation latency.

In all of these methods, draft tokens are verified using rejection sampling, which preserves lossless sampling from the target model distribution [1], [2].

Table I
Accepted length for different drafters on math benchmarks. K = 7. Full table in Appendix A

Drafter	Math		
	GSM8K	Math500	AIME25
EAGLE-3	5.00	4.48	3.81
DFlash	5.39	4.91	4.06
DSpark	**5.93**	**5.82**	**4.93**

Experiments

Experimental Setup

All experiments use the DeepSpec evaluation pipeline [6] with Qwen3-4B [11] as the target model and the drafter checkpoints released with DeepSpec. All three drafters were trained with 7 draft tokens. We evaluate them at $K \in \{4, 7, 8, 12, 16\}$, so every $K \neq 7$ runs a drafter outside its training length. Generation uses standard speculative sampling with temperature 1.0, non-thinking mode and at most 2048 new tokens. Accepted length counts the bonus token emitted by the target model in each verification round. We evaluate on GSM8K [12], Math500 [13] and AIME25 [14], limited to 50 problems for GSM8K and Math500 and all 30 problems of AIME25. Inference runs on a single NVIDIA RTX 3090 (24 GB) with batch size 1 in bf16. Unless stated otherwise, decode latency excludes prompt prefill.

Experimental Results

At the trained draft length ($K = 7$), our measurements reproduce the ranking reported by DSpark [5] on all three math benchmarks: DSpark $>$ DFlash $>$ EAGLE-3 (Table I). Here, accepted length counts the bonus token emitted by the target model in each verification round. Averaged over GSM8K, Math500, and AIME25, DSpark improves accepted length by 26.0% over EAGLE-3 and by 16.6%

over DFlash. The DSpark paper reports corresponding math-only improvements of 22.3% and 16.2% under the same aggregation. Our gain over DFlash therefore closely matches the paper, while our gain over EAGLE-3 is 3.7 percentage points larger. Absolute accepted lengths also agree closely. In both our results and the paper, the advantage of DSpark is smallest on GSM8K and larger on Math500 and AIME25.

Figure 1 extends the comparison to different draft lengths. Two trends are consistent with the proposal-length study reported in DSpark [5]. First, DSpark achieves the highest accepted length at every K and on every dataset. Second, its advantage over DFlash increases with the draft length: on our math benchmarks, the relative improvement grows from 11–18% at $K = 4$ to 22–34% at $K = 16$, while the DSpark paper reports an increase from approximately 16% at $K = 8$ to 30% at $K = 16$ on the math domain.

The absolute behavior at large draft lengths, however, differs substantially. In the DSpark paper, the accepted length continues to increase up to $K = 16$, whereas in our experiments DSpark largely saturates after $K = 8$ and DFlash begins to decline. One possible source of this degradation is extrapolation beyond the training block length. The effect is more pronounced for DFlash, whose draft positions are generated in parallel without causal dependencies. DSpark is substantially more robust at large K, which is consistent with its Markov head providing additional conditioning between draft positions, although our experiments do not isolate this mechanism directly.

Importantly, the training horizon alone cannot explain the discrepancy with the published results. The DSpark paper [5] also states that DFlash and DSpark are trained with a block size of 7, and no separate retraining for larger block sizes is reported. Thus, our results reproduce the qualitative advantage of semi-autoregressive drafting as K increases, but show substantially stronger degradation outside the training horizon. The precise source of this discrepancy remains unclear and may depend on details of the training or evaluation pipeline.

The latency results also differ in magnitude. The paper [5] reports a 0.2–1.3% per-round overhead of DSpark over DFlash at batch size 128, where target verification dominates the total computation. At batch size 1, we measure an overhead that grows from 3.9% at $K = 4$ to 9.6% at $K = 16$. In this regime, the sequential Markov-head computation is no longer amortized by a much more expensive target-model verification step, making its relative latency contribution substantially larger as K increases.

The central question is whether higher acceptance translates into lower decode latency. Figure 2 shows the decode latency per generated token. DSpark is the fastest method at every K and on every dataset. Averaged over the three datasets, its best configuration ($K = 8$) needs 13.4 ms per token, 12% less than DFlash (15.3 ms) and 34% less than EAGLE-3 (20.2 ms).

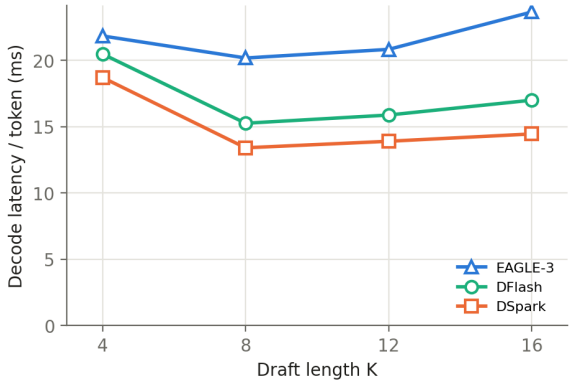


Figure 2. Decode latency per generated token (prefill excluded) as a function of draft length K, averaged over GSM8K, Math500 and AIME25. Lower is better.

The ranking follows from how the cost of each drafter scales with K (Figure 1, right). EAGLE-3 drafts sequentially, so its step latency grows linearly, by about 3.3 ms per additional draft token. Its higher accepted length at large K does not compensate for this, and its per-token latency rises again for $K > 8$. DFlash keeps an almost constant step latency, but its accepted length falls beyond $K = 8$. DSpark pays a small, slowly growing cost for its sequential head and in return keeps most of its accepted length at large K.

For all three methods, the latency optimum averaged over datasets lies at $K = 8$, immediately next to the draft length of 7 tokens used in training. Moving further from this region in either direction increases average latency. At the same time, the latency-optimal K varies across datasets. The DSpark paper [5] reports that acceptance varies strongly across math, code, and chat workloads and uses this observation to motivate a confidence head that adapts the verified length. Our results show that variation in the optimal fixed K is present even across datasets within the math domain.

Resource Efficiency

Figure 3 (top) splits the decode latency per generated token at $K = 7$ into phases. Target verification dominates for every method, at 11.7–14.7 ms per token. Its per-token cost is therefore set by the accepted length: the method that commits more tokens per pass pays less for verification per token. Drafting is where the methods differ structurally. EAGLE-3 spends 5.5 ms per token (26% of decode time) on its seven sequential draft steps. DFlash and DSpark need a single parallel forward, which costs 2.2–2.3 ms per token (14–16%). The drafter state update is negligible, except for EAGLE-3 (0.6 ms), which extends its own KV cache.

The memory picture is reversed (Figure 3, bottom). Additional peak memory over the AR baseline is dominated by drafter weights. EAGLE-3 has 0.93B parameters and

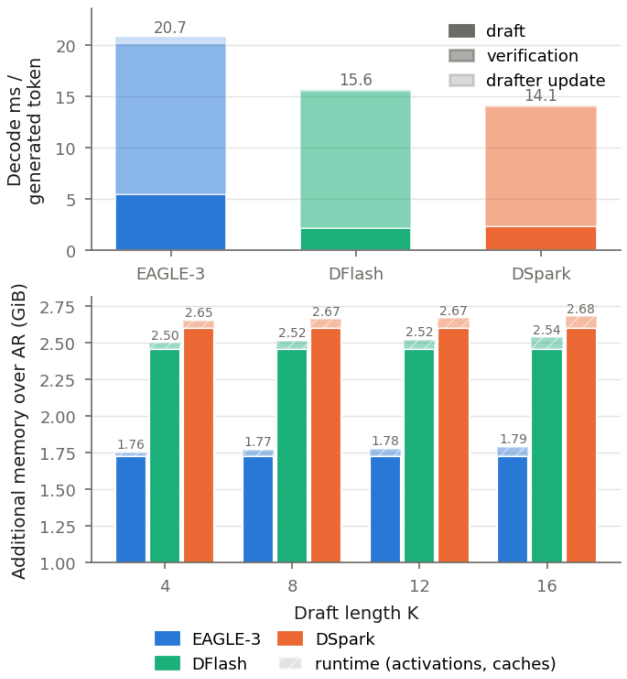


Figure 3. Resource usage. Top: decode latency per generated token at $K = 7$, split into drafting, target verification and drafter update. Bottom: additional peak GPU memory over the autoregressive baseline as a function of K, split into drafter weights and runtime memory.

adds 1.73 GiB, while DFlash (1.32B) and DSpark (1.39B) add 2.46 and 2.60 GiB. Runtime memory (activations and caches) adds only 0.01–0.12 GiB and grows slowly with K. Relative to the 7.7 GiB peak of the AR baseline, DSpark at $K = 8$ therefore obtains its $4.3\times$ speedup with about 34% more memory. DFlash reaches approximately $3.8\times$ with 33% more, and EAGLE-3 reaches $2.9\times$ with 23% more. EAGLE-3 is thus the cheapest drafter in absolute memory cost, while DFlash and DSpark achieve comparable speedup per additional GiB, both outperforming EAGLE-3 on this efficiency measure.

Limitations

Our comparison does not cover all recent speculative decoding methods. In particular, we do not evaluate DFlash2, a recent extension of DFlash that, like DSpark, introduces additional autoregressive dependencies between draft positions, but implements them through a different mechanism while preserving parallel drafting [15].

Another limitation concerns the application of our results to reinforcement-learning rollouts. Our experiments assume a fixed target model and a drafter trained to approximate its distribution. During RL training, however, the target policy continuously changes while the drafter is typically kept fixed, potentially creating an increasing distribution mismatch. Recent work shows that MTP acceptance can degrade substantially during RL and relates this degradation to changes in the entropy of the target policy [16].

Finally, our experimental scope is limited. We evaluate a single target model, Qwen3-4B, on three mathematical reasoning datasets, with GSM8K and Math500 restricted to 50 examples. Moreover, all evaluated drafter checkpoints were trained with a draft length of $K = 7$, whereas we additionally test $K \in 4, 8, 12, 16$. Experiments across more model scales, domains, larger evaluation sets, and drafters explicitly trained for different values of K are required to establish how broadly the observed trends generalize.

Conclusion

We evaluated EAGLE-3, DFlash, and DSpark under a common DeepSpec setup, focusing on how draft length affects acceptance and decoding latency. At the training draft length, our results closely reproduce the previously reported advantage of DSpark over DFlash and EAGLE-3. However, when extending the draft beyond the training horizon, we observe substantially stronger degradation in acceptance than reported in the original DSpark study. We also find that, at batch size 1, DSpark has a larger latency overhead over DFlash than reported in previous experiments with large batch sizes, because the additional sequential computation of its Markov head constitutes a larger fraction of the total decoding cost.

Despite this overhead, DSpark provides the best overall acceptance-latency trade-off and achieves the lowest decode latency per generated token across all evaluated draft lengths. Our experiments also show that the optimal draft length is not universal: the latency optimum remains close to the training horizon on average, while its exact position varies across datasets. These results show that acceptance length alone is insufficient for comparing speculative decoding methods, since higher acceptance may be offset by additional drafting cost. The method and draft length should therefore be selected based on the resulting decode latency under the target inference setting.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *Proc. ICML*, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” arXiv:2302.01318, 2023.

[3] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test,” in *Proc. NeurIPS*, 2025, arXiv:2503.01840.

[4] J. Chen, Y. Liang, and Z. Liu, “DFlash: Block Diffusion for Flash Speculative Decoding,” in *Proc. ICML*, 2026, arXiv:2602.06036.

[5] X. Cheng, X. Yu, C. Shao, J. Li, *et al.*, "DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation," arXiv:2607.05147, 2026.

[6] DeepSeek-AI, "DeepSpec: A Full-Stack Framework for Speculative Decoding," GitHub repository, 2026. [Online]. Available: https://github.com/deepseek-ai/DeepSpec

[7] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Gholami, and K. Keutzer, "Speculative Decoding with Big Little Decoder," in *Proc. NeurIPS*, 2023.

[8] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," in *Proc. ICML*, 2024.

[9] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," in *Proc. ICML*, 2024.

[10] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees," in *Proc. EMNLP*, 2024.

[11] A. Yang *et al.*, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[12] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training Verifiers to Solve Math Word Problems," arXiv:2110.14168, 2021.

[13] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's Verify Step by Step," in *Proc. ICLR*, 2024.

[14] M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev, "MathArena: Evaluating LLMs on Uncontaminated Math Competitions," arXiv:2505.23281, 2025. Dataset: https://huggingface.co/datasets/MathArena/aime_2025

[15] Inco AI, "DFlash 2: Keep Drafting Parallel," blog post, Aug. 2026. [Online]. Available: https://inco.ai/blog/dflash2/

[16] Y. Li, H. Jiang, Y. Xu, *et al.*, "Breaking Entropy Bounds: Accelerating RL Training via MTP with Rejection Sampling," arXiv:2606.12370, 2026.

Appendix A.
Additional Results

Table II
Accepted length for different drafters
on math benchmarks at draft lengths $K \in \{4, 8, 12, 16\}$ (incl. bonus token).

K	Drafter	Math		
		GSM8K	Math500	AIME25
4	EAGLE-3	3.97	3.64	3.32
	DFlash	3.92	3.70	3.23
	DSpark	**4.36**	**4.12**	**3.81**
8	EAGLE-3	5.35	4.71	3.84
	DFlash	5.74	4.98	4.10
	DSpark	**6.45**	**5.99**	**5.22**
12	EAGLE-3	6.18	5.19	4.17
	DFlash	5.63	4.80	3.92
	DSpark	**6.68**	**5.98**	**4.90**
16	EAGLE-3	6.34	5.08	4.16
	DFlash	5.41	4.56	3.66
	DSpark	**6.58**	**5.89**	**4.90**

Table III
Decode latency per step and per token (ms)
for different drafters on math benchmarks at draft lengths $K \in \{4, 7, 8, 12, 16\}$.

K	Drafter	Latency / decode step (ms)			Latency / token (ms)		
		GSM8K	Math500	AIME25	GSM8K	Math500	AIME25
–	AR (Qwen3-4B)	57.70	57.71	57.87	57.70	57.72	57.89
4	EAGLE-3	78.95	79.08	79.24	19.89	21.74	23.92
	DFlash	**73.58**	**73.54**	**73.56**	18.79	19.87	22.82
	DSpark	76.45	76.32	76.36	**17.55**	**18.51**	**20.09**
7	EAGLE-3	91.06	91.21	89.98	18.20	20.38	23.65
	DFlash	**73.99**	**73.80**	**73.69**	13.73	15.03	18.16
	DSpark	77.96	77.74	77.61	**13.15**	**13.37**	**15.76**
8	EAGLE-3	91.68	91.77	91.75	17.15	19.50	23.92
	DFlash	**74.18**	**73.96**	**73.82**	12.93	14.86	18.01
	DSpark	78.45	78.21	78.19	**12.16**	**13.07**	**15.01**
12	EAGLE-3	104.96	105.15	105.13	16.97	20.28	25.27
	DFlash	**74.63**	**74.36**	**74.11**	13.25	15.49	18.93
	DSpark	80.40	80.03	79.77	**12.04**	**13.38**	**16.30**
16	EAGLE-3	118.94	119.38	119.30	18.77	23.52	28.69
	DFlash	**75.70**	**75.29**	**75.05**	13.98	16.53	20.51
	DSpark	82.81	82.50	82.18	**12.59**	**14.02**	**16.78**